Architecture des ordinateurs
Corrigé du TD 2 : Arithmétique des ordinateurs et codage

1. Donner la valeur décimale des entiers suivants, la base dans laquelle ces entiers sont codés étant précisée.
   (a) 1011011 et 101010 en binaire (base 2) ;

   Correction : $1011011_2 = 91_{10}$, $101010_2 = 42_{10}$.

   (b) A1BE et C4F3 en hexadécimal (base 16) ;

   Correction : $A1BE_{16} = 41\ 406_{10}$, $C4F3_{16} = 50\ 419_{10}$.

   (c) 77210 et 31337 en octal (base 8).

   Correction : $77210_8 = 32\ 392_{10}$, $31337_8 = 13\ 023_{10}$.

2. Coder l'entier 2 397 successivement en base 2, 8 et 16.

   Correction : $2\ 397_{10} = 100101011101_2 = 4535_8 = 95D_{16}$.

3. Donner la valeur décimale du nombre 10101, dans le cas où il est codé en base 2, 8 ou 16.

   Correction : $10101_2 = 21_{10}$, $10101_8 = 4\ 161_{10}$, $10101_{16} = 65\ 793_{10}$.

4. Combien d'entiers positifs peut-on coder en binaire sur un octet ?

   Correction : *Un octet contient 8 bits, on peut donc coder* $2^8 = 256$ *entiers.*

   Combien de bits faut-il pour représenter 65 563 entiers différents en binaire ?

   Correction : *Avec b bits, on peut coder* 2^b *entiers différents. Pour coder n entiers, il nous faut donc m bits tels que* $2^{m-1} < n \leq 2^m$, *c.-à-d.* $m - 1 < \log_2 n \leq m$. *On a donc* $m = \lfloor \log_2 n \rfloor + 1$.
   Pour $n = 65\ 563$, *on a* $m = \lfloor \log_2 65\ 563 \rfloor + 1 = 17$.

5. Coder en binaire sur un octet les entiers 105 et 21 puis effectuer l'addition binaire des entiers ainsi codés. Vérifier que le résultat sur un octet est correct. Même question avec les entiers 184 et 72.

Correction :

	1101001	(105)
+	10101	(21)
	1111110	(126)

Ce résultat est correct.

	10111000	(184)
+	1001000	(72)
	(1)00000000	(0)

Ce résultat n'est pas correct (sur 8 bits).

6. Coder en binaire sur un octet les entiers 79 et 52 puis effectuer la multiplication binaire des entiers ainsi codés. Même question avec les entiers 135 et 46.

Correction :

	1001111	(79)
×	110100	(52)
	1001111	
	1001111	
+	1001111	
	1000000001100	(4108)

	10000111	(135)
×	101110	(46)
	10000111	
	10000111	
	10000111	
+	10000111	
	1100001000010	(6 210)

7. Indiquer la valeur codée par le mot de 16 bits 1101100101110101 suivant qu'il représente un entier non signé, ou un entier signé.

Correction : *En non signé, la valeur est* $1101100101110101_2 = 55\,669_{10}$. *En signé, le premier bit (bit de signe) vaut 1, c'est donc un nombre négatif dont la valeur est* $-101100101110101_2 = -22\,901_{10}$.

Même question avec le mot 1001000011101101.

Correction : *En non signé, la valeur est* $1001000011101101_2 = 37\,101_{10}$. *En signé, c'est un nombre négatif dont la valeur est* $-1000011101101_2 = -4\,333_{10}$.

8. Coder en IEEE 754 les nombres 25.875 et 142.0, puis la somme 25.875 + 142.0 et la différence 25.875 - 142.0.

Encodage de 25.875 – Version 1 :

25 > 0 donc s = 0

25 = 16+8+1 = $2^4 + 2^3 + 2^0$ = 0b 11001

.875 = 0.5 + 0.25+ 0.125 = $2^{-1} + 2^{-2} + 2^{-3}$ = 0b111

*25.875 = 0b 11001.111 = 0b 1.1001111 * 2^4*

Exposant = 127 + 4 = 131 = 128 + 2 + 1 = $2^7 + 2^1 + 2^0$ = 0b 1000 0011

Mantisse = 0b 1001 1110 0…0

25.875 = 0b 0100 0001 1100 1111 0000 0000 000 0000 = 0x 41CF 0000

Encodage de 25.875 – Version 2 :

*25.875 = 1.6171875 * 16 = 1.6171875 * 2^4*

Exposant = 127 + 4 = 131 = 128 + 2 + 1= $2^7 + 2^1 + 2^0$ = 0b 1000 0011

.6171875 = $2^{-1} + 2^{-4} + 2^{-5} + 2^{-6} + 2^{-7}$ = 0b 1001 1110 0…0

25.875 = 0b 0100 0001 1100 1111 0000 0000 000 0000 = 0x 41CF 0000

Encodage de 142.0

142 > 0 donc s = 0

142 = 128 + 8 + 4 + 2 = $2^7 + 2^3 + 2^2 + 2^1$ = 0b 1000 1110

*142 = 0b 1.0001110 * 2^7*

Exposant = 127 + 7 = 134 = 128 + 4 + 2 = $2^7 + 2^2 + 2^1$ = 0b 1000 0110

Mantisse = 0b 0001 1100 0…0

142.0 = 0b 0100 0011 0000 1110 0000 0000 0000 0000 = 0x 430E 0000

Encodage de 25.875 + 142.0

Somme de nombres positifs donc s = 0

25.875+142.0 = 167,875

*25.875 = 0b 1.1001111 * 2^4*

*142 = 0b 1.0001110 * 2^7*

Il faut aligner l'exposant le plus petit en le multipliant par 2^3 (décalage de 3 bits vers la gauche) et donc diviser sa mantisse par 2^{-3} (décalage vers la droite de 3 bits)

*25.875 = 0b 1.1001111 * 2^4 = 0b 0.0011001111 * 2^7*

On peut ensuite additionner les mantisses

0b 0.001 1001 1110

+ 0b 1.000 1110 0000

= 0b 1.010 0111 1110

La mantisse est déjà normalisée (1.xxx) donc l'exposant reste inchangé (=7)

Mantisse = 0b 0100 1111 1100 0…0

Exposant = 127 + 7 = 134 = 0b 1000 0110

25.875+142.0 = 0b 0100 0011 0010 0111 1110 0000 0000 0000 = 0x 4327 E000

Encodage de 25.875 - 142.0

Différence de 2 nombres avec résultat négatif donc s = 1

Démarche identique pour le calcul. Après alignement des mantisses, il suffit de faire la différence.

Version 1 : Comme le résultat de 25.875 – 142.0 est négatif, il est plus simple de faire 142.0 – 25.875 car l'encodage sera le même (seul le signe s change)

0b 1.000 1110 0000

- 0b 0.001 1001 1110

= 0b 0.111 0100 0010

*On normalise 0b 0.111 0100 0010 = 0b 1.110 1000 0100 * 2^{-1}*

Exposant = 127 + 7 + -1= 133 = 0b 1000 0101

25.875-142.0 = 0b 1100 0010 1110 1000 0100 0000 0000 0000 = 0x C2E8 4000

Version 2 du calcul de la mantisse : Comme le résultat de 25.875 – 142.0 est négatif, on peut faire l'addition de 25.875 avec le complément à 2 de 142.0 (c'est ce qui se passe toujours en matériel : A – B = A + (B XOR 0xFFFFFFFF) + 1)

0b -1.000 1110 0000 = 0b 0.111 0001 1111 + 1 = 0b 0.111 0010 0000

0b 0.001 1001 1110

+ 0b 0.111 0010 0000

= 0b 1.000 1011 1110

Le résultat de 25.875 – 142.0 est négatif (MSB = 1) donc on refait le complément à 2 avant de pouvoir faire l'encodage IEEE754

0b 1.000 1011 1110 = 0b 0.111 0100 0001 + 1 = 0b 0.111 0100 0010

Version 3 du calcul de la mantisse : On fait la soustraction directe

0b 0.001 1001 1110

- 0b 1.000 1110 0000

= 0b 11.000 1011 1110

Comme le nombre de 25.875 – 142.0 est négatif (MSB = 1), on doit calculer son complément à 2 avant encodage.

0b 11.000 1011 1110 -> 0b 00.111 0100 0001 + 1 = 0b 0.111 0100 0010